PAALHU: project status and pending work

Restaurant website with bookings and orders · 22 October 2026 · Based on the "From Screens to Endpoints" worksheet

1. Summary

Part	Status	Notes
Public screens (home, menu, set menu, spice guide, opening hours, location, about us)	Done	Static HTML, no server data needed
Chat widget on the website	Built, switched off	No AI model connected yet (`CHAT_ENABLED = false`)
Booking logic (model)	Mostly done	In `reservas-core.mjs`, stored in a JSON file, not SQLite
Validators	Written	All in `src/validators.js`. No tests yet; not all of them are used yet
Endpoints	2 of ~19	Only `GET /api/chat/status` and `POST /api/chat`
Orders (model)	Not started	Only a script that prices an order; nothing saves orders
Staff screens (login, bookings, orders)	Not started	
Staff login	Not started	
Express + `routes/` folder	Not started	The server uses plain Node `http`
Pagination and filters	Not started	The `page` and `limit` validators already exist
Tests	Not started	

> **Agreed decisions:** customers can book and order through the chat, by phone (+34 627 41 09 35) or by email (owners@paalhu.es). The chat can only do what staff can do by hand. Staff register phone and email bookings and orders through a staff web page with a login. Data will be stored in SQLite (built into Node 24, nothing to install). Validators only say what is wrong and in which field; the controllers choose the HTTP status code (400, 401, 404…).

2. What is already done

Screens

- All public pages in `pages/`, listing chat, phone and email for bookings and orders.
- Chat widget (`js/chat.js`) on the home and booking pages.

Booking model (`reservas-core.mjs`)

- Check availability, with alternatives when a shift is full.
- Create, change and cancel bookings; automatic table assignment, including joining adjacent tables.
- Waiting list, shift occupancy, booking sheet, search.
- Permanent history of every change (`reservas-historial.jsonl`).
- Used by both the chat and the staff command-line tool (`reservas.mjs`).

Entities

Entity	Status	Where
Booking, WaitingListEntry	Done	`reservas-core.mjs` → `reservas.json`. Channel stored only as `chat` / `equipo`
HistoryEvent	Done	`reservas-historial.jsonl`
Table, Shift	Done	`config.json`
Dish	Partial	Only as text in `reference.md` and the menu HTML, no ids
Order, OrderItem	Missing	
Staff	Missing	

Validators (`src/validators.js`)

Each validator returns the clean value or throws a `ValidationError` with a readable message and the name of the wrong field. They contain no HTTP status codes.

Area	Validators
Bookings	`date`, `time`, `guests`, `name`, `phone`, `notes`, `table`, `reference`
Orders	`dish`, `drink`, `quantity`, `price`, `spiceLevel`, `items`, `type`, `address`, `orderTime`, `orderStatus`, `statusChange`
Shared	`channel`, `login`, `page`, `limit`

- The chat uses `date`, `guests`, `name` and `phone`; the order-pricing script uses `dish`, `price` and `quantity`. The rest are ready for the new models and controllers.
- The booking model also checks: time inside a shift, 12-guest maximum, adjacent tables, whether the booking fits.

3. What we need to do, in order

Step 1 · Express and project structure

- Install Express. Split `src/server.js` into `server.js` (starts the app) and `app.js` (exports it).
- Create `src/routes/`, `src/controllers/`, `src/models/`.
- Add `.env` (git-ignored) and `README.md`.
- One error handler in `app.js`: `ValidationError` → 400, no login → 401, not found → 404, no room or status not allowed → 409, anything else → 500.

Step 2 · SQLite database

- Schema (migration) with tables: bookings, waiting_list, orders, order_items, dishes, staff, history.
- Move the existing data from `reservas.json` and the history file into the database.
- Rewrite the storage part of `reservas-core.mjs` (the booking logic stays the same).
- Store the channel as `chat`, `phone` or `email`.
- Add the database file to `.gitignore`: it holds customers' personal data.

Step 3 · Models

- `models/` for Booking and WaitingListEntry (wrapping the booking engine).
- New Order model: create, list, get one, change status. The total is calculated on the server from dish prices.
- Dish model: menu dishes with ids and prices, read-only.
- Staff model: username and hashed password.

Step 4 · Use the validators everywhere

- Chat: also check `time`, `notes` and `reference`.
- Staff bookings through `reservas.mjs`: use the same validators.
- New controllers: validate every request body and query with `validators.js` before calling a model.

Step 5 · Controllers and routes

Controller	Endpoints
`authController`	`POST /api/auth/login`, `POST /api/auth/logout`
`bookingController`	`GET /api/shifts`, `GET /api/availability`, `GET /api/bookings`, `GET /api/bookings/:id`, `POST /api/bookings`, `PATCH /api/bookings/:id` (change or cancel)
`waitingListController`	`GET /api/waiting-list`, `POST /api/waiting-list`, `PATCH /api/waiting-list/:id`
`orderController`	`GET /api/orders`, `GET /api/orders/:id`, `POST /api/orders`, `PATCH /api/orders/:id`
`dishController`	`GET /api/dishes` (public)

`chatController`	Already exists: `GET /api/chat/status`, `POST /api/chat`

Cancelling uses `PATCH` with `status: "cancelled"`, not `DELETE`: bookings and orders are never deleted.

Step 6 · Staff login

- Hashed passwords and a session or token.
- Every staff endpoint answers 401 without a valid login.

Step 7 · Staff pages

- `/admin/login`: login form.
- `/admin/bookings`: day's shifts and occupancy, bookings, waiting list; forms to create, change and cancel.
- `/admin/orders`: list of orders, form to create an order, buttons to change status.

Step 8 · Pagination and filters

- `GET /api/bookings`: `page`, `limit` (20, max 100), `date` / `from` / `to`, `status`, `search`, `channel`, `sort`.
- `GET /api/orders`: `page`, `limit`, `date`, `status`, `channel`, `type`, `sort`.
- `GET /api/waiting-list`: `date`, `status`.
- Paginated answers return `{ data, page, limit, total }`.

Step 9 · Chat orders

- Add a `create_order` tool to the chat that uses the Order model.
- Remove the current rule that tells customers to call to confirm their order.
- Connect an AI model so the chat is switched on.

Step 10 · Tests

- Validators and the Order model, with Node's built-in `node --test`.

4. Open points to decide

Point	Detail
Maximum guests	`validators.js` accepts 1–14, but the restaurant's maximum is 12. Change the validator to 12.
Same-day bookings	The `date` validator rejects today, so “a table for tonight” fails. The worksheet says only “not in the past”. Decide which rule applies.
Opening hours in two places	`orderTime` keeps its own copy of the opening hours; `reference.md` has the other. Both must change together, or move them into `config.json`.
File rename	`order_total.mjs` was renamed to `order_total_validator.mjs`, but `SKILL.md`, `reference.md` and the script's own usage text still use the old name.
Uncommitted changes	The new validators in `src/validators.js` and this document are not committed yet.

Customer self-service	Customers manage bookings only through the chat. If the course requires it, add public endpoints that check reference + phone.